

The background of the slide features a complex network diagram with numerous nodes and connecting lines, rendered in a light blue color against a dark blue background. The nodes are small squares, and the lines are thin, creating a web-like structure that fills the entire slide.

Monitorización y Auditoría de **Contenedores**

Sesión 5

Introducción

Objetivos de la sesión:

- Integrar herramientas SAST y DAST en tuberías CI/CD.
- Implementar controladores de acceso (RBAC) en entornos cloud.
- Establecer políticas de revisión/aprobación de imágenes de contenedores.
- Monitorear tuberías para detectar fallos de seguridad.

¿Por qué es crucial?

- *Cita clave:* "La monitorización reduce costos y riesgos al detectar vulnerabilidades temprano" (OWASP, 2023).

SAST vs. DAST

Tabla comparativa:

SAST (ANÁLISIS ESTÁTICO)	DAST (ANÁLISIS DINÁMICO)
<i>Fase temprana (código fuente)</i>	<i>Fase tardía (aplicación en ejecución)</i>
Detecta errores de codificación	Simula ataques reales (ej: SQLi, XSS)
Herramientas: SonarQube, Checkmarx	Herramientas: OWASP ZAP, Burp Suite

Ejemplo práctico:

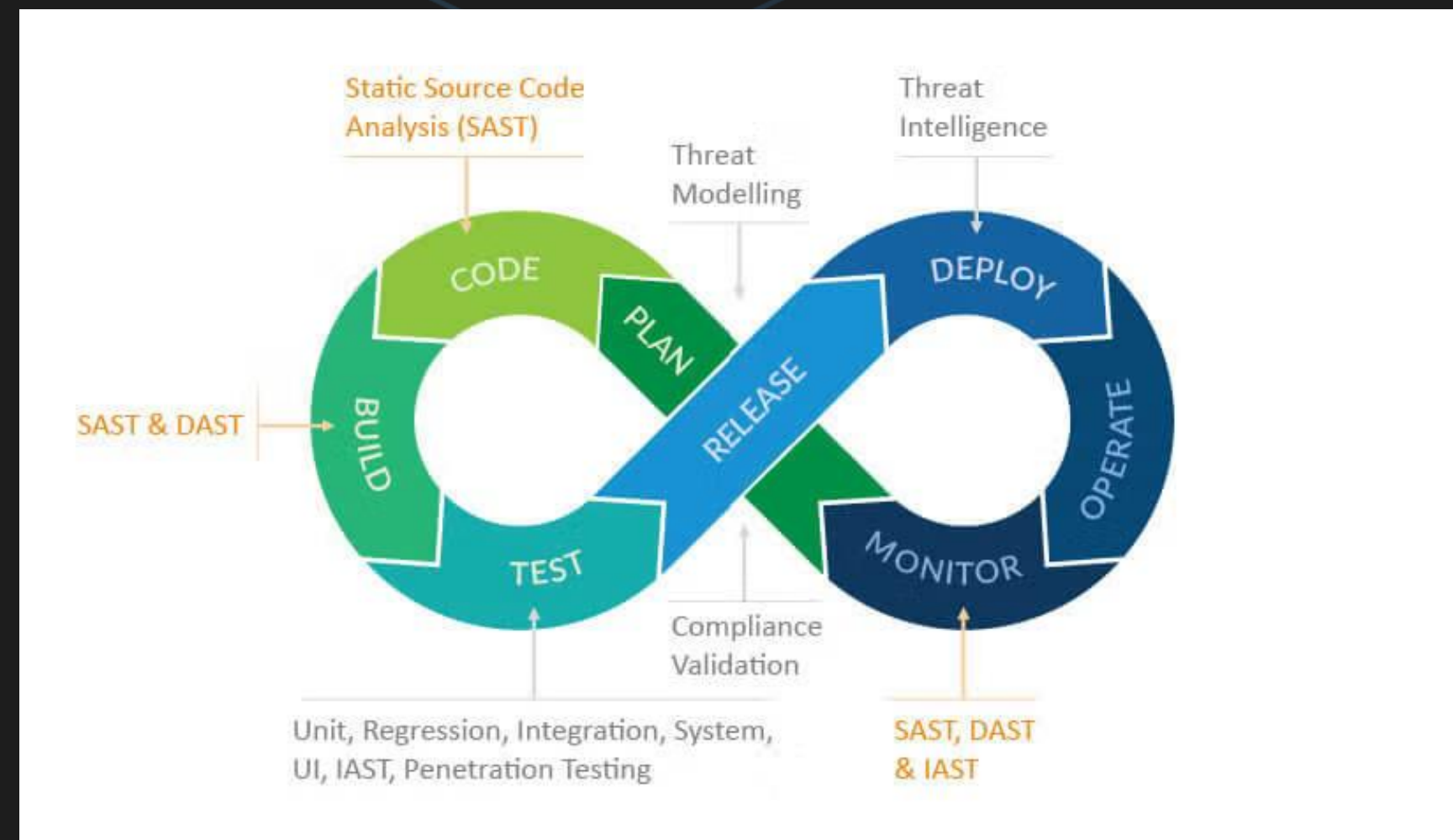
- SAST: Integrar SonarQube en GitLab CI/CD (script YAML).
- DAST: Escanear con OWASP ZAP en Jenkins (archivo).

zap-report.html

Integración de SAST en CI/CD

- **Pasos Clave:**

1. Instalar herramienta (ej: SonarQube en Docker).
2. Configurar trabajo de análisis en el pipeline.
3. Definir umbrales (ej: bloquear construcción si >5 vulnerabilidades críticas).
4. Generar informes (HTML).



Control de Acceso (RBAC)

- **Autenticación:** LDAP, OAuth, MFA.
- **Autorización:** Roles (desarrollador, tester, admin) y permisos mínimos.

Ejemplo en Jenkins:

- Complemento *Control de acceso basado en roles* .

- Asignar permisos: , , . `job-read` `job-build` `admin`

- **Ejemplos en Kubernetes:**

```
1 # Rol para desarrolladores
2 apiVersion: rbac.authorization.k8s.io/v1
3 kind: Role
4 metadata:
5   namespace: ci-cd
6   name: developer-role
7 rules:
8 - apiGroups: [""]
9   resources: ["pods", "services"]
10  verbs: ["list", "get"]
```

Políticas de Revisión de Imágenes

Componentes:

1. Escaneo de vulnerabilidades (Trivy, Anchore).
2. Validación de configuraciones (ej: reducir superficie de ataque).
3. Firma de imágenes (garantizar integridad).

GitOps con Argo CD:

- Flujo: Repositorio Git → Argo CD → Kubernetes.
- Política de aprobación: Solo imágenes con etiqueta .

.approved

Monitoreo de Tuberías

Herramientas Clave:

- **Prometheus + Grafana:** Métricas en tiempo real (tiempo de ejecución, fallos).
- **Falco:** Detección de actividades sospechosas (ej: acceso no autorizado).
- **Loki + Grafana:** Análisis de registros.

Ejemplo de Alerta en Prometheus:

```
1 groups:
2 - name: pipeline-alerts
3   rules:
4 - alert: PipelineFailure
5   expr: job_failures > 5
6   for: 5m
7   labels: {severity: "critical"}
```

Detección de Accesos No Autorizados

Estrategias:

•Regla Falco:

```
1 rule: Unauthorized Pipeline Access
2 desc: Detecta acceso de usuarios no autorizados
3 condition: user.uid != 1000 and proc.name = "pipeline"
4 output: Alerta: Acceso sospechoso (user=%user.uid)
```

Análisis de Logs: Usar ELK Stack o Loki para identificar patrones anómalos.

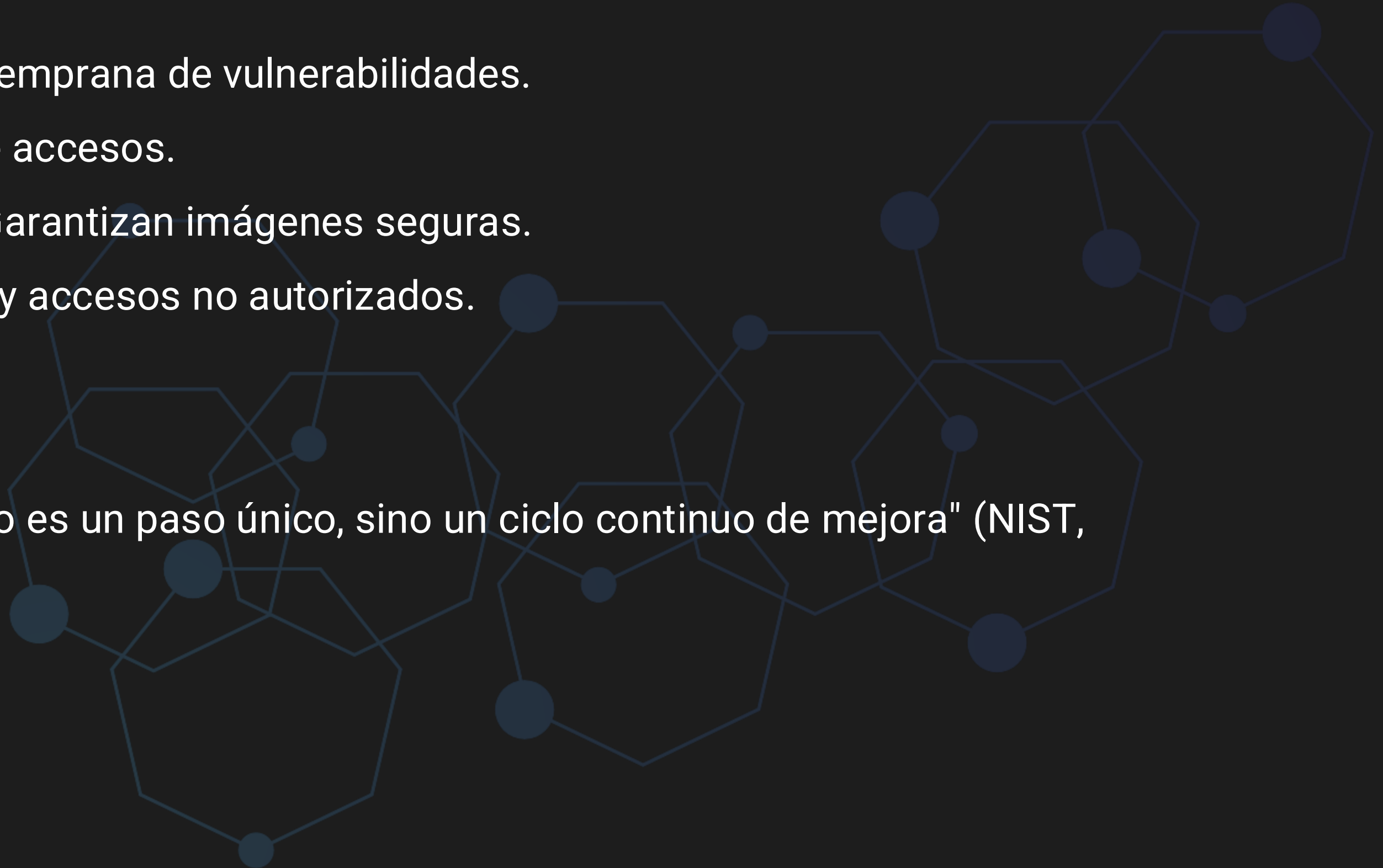
Conclusión

Resumen:

- SAST/DAST: Prevención temprana de vulnerabilidades.
- RBAC: Control granular de accesos.
- Políticas de aprobación: Garantizan imágenes seguras.
- Monitoreo: Detecta fallos y accesos no autorizados.

Reflexión Final:

"La seguridad en la nube no es un paso único, sino un ciclo continuo de mejora" (NIST, 2022).



Referencias

- OWASP. (2023). *Guía de Pruebas de Seguridad* .
- SónarQube. (2023). *Análisis Estático de Código* .
- NIST. (2022). *Guía de Seguridad de Contenedores* .
- Kubernetes. (2023). *Documentación de RBAC* .